栈内存 vs 堆内存：核心区别精讲

栈（Stack）和堆（Heap）是程序**运行时内存**的两大核心区域，二者的本质差异围绕**管理方式、速度、生命周期、空间、线程属性**展开，也是C/C++、Java等语言内存模型的基础。

下面直接抓**核心区别**，用最关键的维度对比：

一、核心区别对比表

对比维度	栈内存（Stack）	堆内存（Heap）
分配/释放	自动分配、自动释放（编译器/系统接管）	手动分配、手动释放（C++:new/malloc；free/delete）
分配速度	极快（仅移动栈指针）	慢（需遍历空闲块、分配算法）
空间大小	小且固定（通常几MB），易栈溢出	大且灵活（受限于虚拟内存）
内存连续性	连续分配，**无内存碎片**	离散分配，频繁操作易产生**内存碎片**
生命周期	随函数/代码块结束立即销毁	由程序员控制，直到手动释放/程序终止
线程属性	**线程私有**，每个线程独立栈	**进程共享**，多线程访问需加锁
存储内容	局部变量、函数参数、返回地址	动态对象、大数据、需长期存活的数据

二、通俗理解

- **栈**：像**抽屉**，空间小、拿取极快，用完自动归位，不用自己管。

函数调用时入栈，执行完直接弹栈，变量自动销毁。

- **堆**：像**大仓库**，空间极大，但要自己找位置、用完自己清理，找位置慢，还容易乱（碎片）。

三、C++ 典型示例

Code block

```
void func() {
    // 栈内存：自动分配，函数执行完自动释放
    int num = 10;
    char arr[10];

    // 堆内存：手动new分配，必须手动delete释放
    int* p = new int(20);
    // 不delete会造成内存泄漏
    delete p;
}
```

四、关键易错点

1. **栈溢出**：递归过深、超大局部数组，会直接爆栈（栈空间太小）。
2. **内存泄漏**：堆内存分配后忘记释放，程序运行越久占用内存越多。
3. **野指针**：堆内存释放后，指针未置空，再次访问会出错。
4. 栈变量**不能跨函数使用**，堆对象可以全局/跨函数存活。

（注：文档部分内容可能由 AI 生成）